

MagicalBrush 影视创作平台 — 架构决策与实施方案 v2

MagicalBrush 影视创作平台 — 架构决策与实施方案 v2

日期：2026-07-15 基于：plan20260708.md (Baizor-API + Hermes-Agent 方案) 焦点：huayu-drama-4 全模态能力确认，混合架构设计，配置建议

〇、当前进展与服务器实况

0.1 Baizor-API 侧已完成

已完成项	状态	位置
huayu-drama-4 注册为图像生成模型	☑	common/model.go ImageGenerationModels
Image Playground 前端	☑	web/default/src/features/image-playground/
Image Playground 路由	☑	/pg/images/generations + /api/user/image-playground/*
默认模型设为 huayu-drama-4	☑	constants.ts, index.tsx defaultModel
图像生成历史（服务端持久化）	☑	controller/image_playground_history.go
视频生成 API 路由	☑	router/video-router.go (/v1/video/generations, /v1/videos)
7+ 视频供应商适配器	☑	relay/channel/task/ (kling, sora, vidu, hailuo, doubao, ali, gemini)
HermesExecutionTask 长任务模型	☑	model/hermes_execution_task.go
Hermes Playground 前端	☑	web/default/src/features/hermes-playground/
BillingSession 全生命周期计费	☑	service/billing_session.go

0.2 anne-1 服务器实况 (8× RTX 4090 24GB = 192GB)

通过 ssh anne-1 查看 /lucky/huazhen/docker-compose.yml，确认如下部署：

llama-proxy-huazhen (:8080) 统一入口 - 按路径路由到不同后端
/v1/chat/completions → llama-huazhen (文本) 或 llama-v1-huazhen (含图片时自动切换)
/v1/images/generations → ideogram4-huazhen
/v1/videos/generations → sulphur2-huazhen → comfyui-huazhen
/v1/models → 静态列表

服务清单

容器	功能	GPU	模型/引擎	对外别名
llama-huazhen	文本 Chat	GPU 0,1,2,3 (4 卡)	Qwable-v1 (IQ4_XS, 262K ctx)	qwable-v1, huazhen-v1, huazhen-fable-5
llama-v1-huazhen	视觉理解	GPU 6 (1卡)	Qwen3-VL-30B-A3B (IQ4_XS)	qwen3-v1-30b-a3b, huazhen-v1
ideogram4-huazhen	图像生成	GPU 7 (1卡)	Ideogram 4 (nf4量化)	ideogram-4
comfyui-huazhen	ComfyUI 后端	GPU 4,5 (2卡)	LTX 2.3 / Sulphur 2	— (内部)
sulphur2-huazhen	视频生成 API	— (CPU, 调 ComfyUI)	Sulphur 2 (fp8)	sulphur-2
llama-proxy-huazhen	统一代理	— (CPU)	—	对外 :8080

GPU 分配现状

GPU 0-3 (4× 24GB = 96GB) → llama-huazhen (文本LLM, 4卡tensor split)
GPU 4-5 (2× 24GB = 48GB) → comfyui-huazhen (视频生成后端)
GPU 6 (1× 24GB = 24GB) → llama-v1-huazhen (视觉理解)
GPU 7 (1× 24GB = 24GB) → ideogram4-huazhen (图像生成)

视频生成当前参数

参数	当前值	说明
最大分辨率	1280px (长边)	SULPHUR2_MAX_SIDE=1280
默认尺寸	512×768	DEFAULT_SIZE
帧数	97帧	SULPHUR2_NUM_FRAMES=97
帧率	24fps	→ 约 4 秒视频
推理步数	50步	NUM_INFERENCE_STEPS=50
Guidance Scale	3.6	
最大并发	1 (同步阻塞)	sulphur2 单请求阻塞等待 ComfyUI
超时	3600s (1小时)	PROXY_UPSTREAM_TIMEOUT

一、Q1-Q5 确认结果

#	问题	确认答案
Q1	huayu-drama-4 支持哪些能力？	☒ 全模态 ：chat + image + image-vl + video，由 llama-proxy-huazhen 统一代理，按路径路由到 4 个后端
Q2	视频生成是异步还是同步？	☒ 异步 。sulphur2 本身是同步阻塞（等 ComfyUI 完成），但 Baizor-API 侧应通过 Task 系统异步封装
Q3	视频 API 格式？	☒ 已兼容 OpenAI Videos API ：POST /v1/videos/generations，请求 {prompt, size, n, ...}，响应 {created, data: [{b64_json}]}
Q4	支持 image-to-video？	☒ 当前不支持，可在服务器扩展 。sulphur2 当前仅 text-to-video，i2v 需要扩展 ComfyUI 工作流
Q5	最大配置建议？	☒ 见下方”最高配建议”

二、核心决策：混合架构（A + B 分层）

结论：不必二选一

经评估，**推荐 HermesAgent + 直接 LLM/API 调用分层混合**：

维度	路径 A: HermesAgent	路径 B: 直接 LLM	推荐
剧本拆解	Skill 定义 prompt + 输出格式, Agent 自动重试/修正	Go 代码硬编码 prompt, 需自行处理重试和格式校验	A
多步编排	Agent 原生支持: 拆解 → 规划 → 生成 → 校验 → 修复, 全自动	需 Go 代码编排 pipeline, 每步写状态机	A
工具调用	109+ 已有工具 + FFmpeg terminal + 图像/视频 API	只能调 Baizor-API REST 端点, FFmpeg 需额外服务	A
角色一致性	Agent 可跨镜头记忆上下文, 自主决策是否需要修复	需 Go 代码显式传递上下文, 逻辑复杂	A
异常处理	Agent 可解释错误、自动调整参数重试	硬编码错误处理分支	A
实时交互	Hermes WebSocket 流式输出, 前端已有 Playground	需新建 SSE/WS 接口	A
延迟	Agent 启动 + LLM 多轮 ≈ 20-60s 额外开销	单次 LLM 调用 ≈ 3-10s	B
简单任务	小题大做: 单张图生成不需要 Agent	直接调 API, 毫秒级	B
可控性	Agent 行为有随机性, prompt 调优周期长	Go 代码完全确定性	B
部署复杂度	需 Python + Hermes 运行时	纯 Go, 单二进制	B

分层架构

用户操作	
简单操作 (Go 后端直接调用, ≤1步)	
└ 单张图像生成	→ /v1/images/generations
└ 单段视频生成	→ /v1/videos/generations (Task异步)
└ 图像理解/描述	→ /v1/chat/completions (VL模型)
└ 剧本快速拆解	→ /v1/chat/completions (structured)
复杂编排 (HermesAgent, ≥2步 + 需决策)	
└ 剧本深度拆解+分镜规划	→ script-analyzer Skill
└ 多镜头批量生成	→ batch-generator Skill
└ 角色一致性校验+修复	→ consistency-guard Skill
└ 后期合成 (FFmpeg)	→ post-production Skill
└ 端到端全流程	→ full-pipeline Skill
统一基础设施 (Baizor-API)	
└ 计费 (BillingSession)	
└ 任务队列 (Task + TaskPolling)	
└ 通道路由 (channel_select → llama-proxy-huazhen)	
└ 资产管理 (StudioAsset)	

三、Baizor-API 配置建议

3.1 模型注册 (common/model.go)

现状: huayu-drama-4 仅在 ImageGenerationModels 中。

建议: huayu-drama-4 是全模态的统一品牌名。但底层实际是 4 个不同模型, 建议在 Baizor-API 注册多个模型别名, 映射到同一个 Channel (llama-proxy-huazhen), 由 proxy 按路径自动路由。

```
// common/model.go - 修改建议

var (
    ImageGenerationModels = []string{
        "dall-e-3",
        "dall-e-2",
        "gpt-image-1",
        "gpt-image-2",
        "huayu-drama-4",           // ← 保留，图像生成
        "prefix:imagen-",
        "flux-",
        "flux.1-",
    }

    // 新增：视频生成模型列表
    VideoGenerationModels = []string{
        "huayu-drama-4",           // 同名，由请求路径区分
        "huayu-drama-4-video",     // 或显式的视频模型别名
    }
)

// 新增
func IsVideoGenerationModel(modelName string) bool {
    modelName = strings.ToLower(modelName)
    for _, m := range VideoGenerationModels {
        if strings.Contains(modelName, m) {
            return true
        }
    }
    return false
}
```

3.2 Channel 配置（管理后台）

在 Baizor-API 管理后台添加 Channel：

通道名称：huazhen-cluster
 类型：OpenAI（兼容）
 Base URL：http://<anne-1-ip>:8080 ← llama-proxy-huazhen 对外端口
 API Key：<LLAMA_API_KEY>
 模型列表：huayu-drama-4, huazhen-v1, huazhen-vl, huazhen-fable-5
 优先级：10
 权重：100
 分组：default

关键：由于 llama-proxy-huazhen 内部按路径路由，一个 Channel 即可覆盖 chat + image + video 所有请求。Baizor-API 只需按 model name 将请求路由到这个 Channel。

3.3 llama-proxy-huazhen 配置建议

3.3.1 增加 huayu-drama-4 到 PROXY_MODELS

当前：

```
- PROXY_MODELS=huazhen-v1,huazhen-vl,ideogram-4,sulphur-2
```

建议：

```
- PROXY_MODELS=huayu-drama-4,huazhen-v1,huazhen-vl,ideogram-4,sulphur-2
```

并在 proxy.py 中将 huayu-drama-4 作为统一别名，所有 4 个能力的请求都接受 model: huayu-drama-4：
 - /v1/chat/completions + model=huayu-drama-4 → 路由到 llama-huazhen
 - /v1/images/generations + model=huayu-drama-4 + 含图片 → 路由到 llama-vl-huazhen
 - /v1/videos/generations + model=huayu-drama-4 → 路由到 ideogram4-huazhen
 - /v1/sulphur2 + model=huayu-drama-4 → 路由到 sulphur2-huazhen

3.3.2 新增异步视频 API 支持

当前问题： sulphur2 是同步阻塞（一个视频生成可能要 5-30 分钟），proxy 直接透传等待。对于 Baizor-API 的 Task 异步系统，建议两种方案：

方案 A（推荐）：proxy 层新增异步封装

在 llama-proxy-huazhen 中新增 OpenAI Videos API 异步模式支持：

```
# proxy.py 新增路由
ALLOWED_PATHS.add("/v1/videos")           # POST 提交 + GET 查询
ALLOWED_PATHS.add("/v1/videos/{id}")       # GET 查询单个

# POST /v1/videos → 提交任务, 立即返回 task_id
# GET /v1/videos/{id} → 查询任务状态/结果
```

请求格式（兼容 OpenAI Videos API）：

```
POST /v1/videos
{
  "model": "huayu-drama-4",
  "input": [{"type": "text", "text": "A cinematic shot of..."}],
  "size": "1280x720",
  "duration": 5
}

→ 202 Accepted
{
  "id": "video_abc123",
  "status": "generating",
  "created_at": 1752600000
}

GET /v1/videos/video_abc123

→ 200 OK (完成时)
{
  "id": "video_abc123",
  "status": "completed",
  "video": {"url": "https://..."},
  "created_at": 1752600000
}
```

方案 B：Baizor-API 侧 Task 封装

不改 proxy，在 Baizor-API 的 Task 系统中异步封装：- POST /v1/video/generations → 创建 Task → 后台调 proxy 的同步 /v1/videos/generations → 完成后更新 Task - 复用现有 relay/channel/task/ 适配器模式

推荐方案 A，因为：1. 与 Baizor-API 现有视频路由 (/v1/videos) 完全兼容 2. proxy 端可管理队列和并发，避免 ComfyUI 被压垮 3. Baizor-API 无需新建适配器

四、最高配建议（Q5）

4.1 视频生成参数 — 最高配

```
# docker-compose.yml - sulphur2-huazhen 环境变量修改
- SULPHUR2_MAX_SIDE=1920      # 当前 1280 → 提升到 1080p
- SULPHUR2_NUM_FRAMES=193     # 当前 97 → 约 8 秒 @24fps
- SULPHUR2_FPS=24              # 保持 24fps（电影标准）
- SULPHUR2_NUM_INFERENCE_STEPS=75 # 当前 50 → 更高质量
- SULPHUR2_GUIDANCE_SCALE=3.6  # 保持不变
```

参数	当前	最高配	影响
MAX_SIDE	1280	1920	支持 1080p (1920×1080)，需更多显存
NUM_FRAMES	97 (~4s)	193 (~8s)	单镜头 8 秒，影视场景够用
FPS	24	24	电影标准帧率
NUM_INFERENCE_STEPS	50	75	更细腻，生成时间 ×1.5
预计单视频耗时	~5-15 min	~15-40 min	GPU 4,5 两卡

注意：1920px + 193 帧可能在 2× RTX 4090 (48GB) 上 OOM。建议：- 先试 1280 + 193 帧（扩时长不扩分辨率）- 再试 1920 + 97 帧（扩分辨率不扩时长）- 最后试 1920 + 193 帧，如 OOM 则降回

4.2 图像生成参数 — 最高配

```
# docker-compose.yml - ideogram4-huazhen 环境变量
- IDEOGRAM4_MAX_SIDE=2048      # 当前已是 2048，保持
- IDEOGRAM4_QUANTIZATION=nf4    # 当前 nf4（单卡 24G 极限）
- IDEOGRAM4_MAGIC_PROMPT=on     # 保持开启 magic prompt
- IDEOGRAM4_FILTER_CHECK=on     # 保持安全水印检测
```

图像已是最高配（2048px, nf4量化在单卡24G极限）。如需更高质量：- 升级到 bf16 量化需 2 卡（48GB），但需改 ideogram4 服务支持多卡

4.3 Image-to-Video (i2v) 扩展方案

目标：支持先生成关键帧图像 → 再转视频，实现影视分镜 pipeline。

方案：扩展 ComfyUI 工作流 + sulphur2 API

Step 1: 新增 i2v ComfyUI 工作流

```
comfyui-workflows/
├─ ltx23_t2v_base_api.json    ← 现有 text-to-video
└─ ltx23_i2v_base_api.json   ← 新增 image-to-video
```

LTX-Video 2.3 原生支持 image-to-video（LTXVImgToVideoInplace 节点），当前工作流中已有该节点（NODE_BYPASS_I2V_22），只是被 bypass 了。

Step 2: 修改 sulphur2/server.py

```
# sulphur2/server.py - 新增 i2v 端点

# 新增路由
ALLOWED_PATHS.add("/v1/videos/image-to-video")

# POST /v1/videos/image-to-video
# {
#   "model": "huayu-drama-4",
#   "image": "data:image/png;base64,...", // 或 URL
#   "prompt": "the character walks forward...",
#   "size": "1280x720",
#   "num_frames": 97
# }
```

核心改动：1. 取消 NODE_BYPASS_I2V_22 的 bypass（设为 false）2. 将输入图片传入 ComfyUI 的 LoadImage 节点 3. 其余参数复用现有 t2v 逻辑

Step 3: proxy 路由更新

```
# docker-compose.yml - llama-proxy-huazhen
- UPSTREAM_VIDEO_URL=http://sulphur2-huazhen:8080
# proxy.py 新增 ALLOWED_PATHS
# "/v1/videos/image-to-video"
```

预计工作量：2-3 天（ComfyUI 工作流已有 i2v 节点，只需开启 + API 封装）

4.4 后续扩展：更多 GPU 时的方案

如果未来有更多 GPU 可用，优先考虑：

优先级	扩展方向	所需资源	效果
P0	视频生成并发	+2 卡 → ComfyUI 第二实例	视频生成并发 ×2
P1	视频分辨率	+2 卡 → ComfyUI 4卡	稳定 1920p + 长时长
P2	图像生成高精度	+1 卡 → ideogram4 双卡 bf16	图像质量提升
P3	LoRA 训练	+2 卡 → 独立训练实例	支持角色 LoRA 训练
P4	文本 LLM 扩容	+2 卡 → 6 卡 tensor split	更大 ctx 或更快推理

五、修订后的模块清单

相比 plan20260708，以下模块有变化（标 **NEW** 为新增，**✎** 为修订，**☑** 为沿用）：

模块 1：Web 工作台 **☑** 沿用

（同 plan20260708 模块 1，无变化）

模块 2：AI 创作引擎 **☑** 混合架构

2A. 快速生成层（Go 后端直接调用）

```
// service/studio_quick_gen.go
type QuickGenService struct{}

// 单张图像 - 直接调 relay /v1/images/generations
func (s *QuickGenService) GenerateImage(ctx, req) (*resp, error)

// 单段视频 - 提交 Task, 异步调 /v1/videos/generations
func (s *QuickGenService) GenerateVideo(ctx, req) (*resp, error)

// 图生视频 - 提交 Task, 异步调 /v1/videos/image-to-video ☑
func (s *QuickGenService) ImageToVideo(ctx, req) (*resp, error)

// 图像理解 - 调 /v1/chat/completions + 图片（路由到 VL 模型）
func (s *QuickGenService) DescribeImage(ctx, req) (*resp, error)

// 剧本快速拆解 - 单次 structured output
func (s *QuickGenService) QuickAnalyzeScript(ctx, script) (*analysis, error)
```

2B. 智能编排层（HermesAgent）

```
hermes-agent/skills/magicalbrush/
├─ script-analyzer.md      剧本 → 结构化场景/镜头/角色列表
├─ shot-planner.md        场景 → 镜头规划 + 景别/运镜选择
├─ character-designer.md   角色 → 参考图 prompt + 一致性参数
├─ batch-generator.md ☑ 批量镜头生成编排（图→视频 pipeline）
├─ consistency-guard.md    多镜头角色一致性校验 + 修复
├─ quality-checker.md      质量评分 + 异常诊断 + 修复建议
├─ post-production.md      素材 → FFmpeg 后期编排
└─ full-pipeline.md ☑ 端到端：拆解→规划→生图→生视频→合成
```

2C. 集成入口 API

```
// 快速生成 - 直接返回/异步Task
POST /api/studio/project/{id}/quick-generate
  body: { type: "image"|"video"|"i2v"|"describe"|"analyze", ... }

// Agent 智能创作 - HermesExecutionTask
POST /api/studio/project/{id}/agent-create
  body: { skill: "script-analyzer"|"batch-generator"|"full-pipeline", ... }
```

模块 3: Skill Registry ☑ 沿用

模块 4: 队列与调度 ☑ 沿用

模块 5: huayu-drama-4 适配器 ☑ 大幅简化

结论：不需要新建 Channel Adapter。

原因：1. 图像生成走 OpenAI Images API → 现有 OpenAI adapter 直接可用 2. 视频生成走 OpenAI Videos API → 走 Task 系统 + 现有 Sora/Kling 适配器模式 3. Chat 走 OpenAI Chat API → 现有 OpenAI adapter 4. 所有请求由 llama-proxy-huazhen 内部路由，对 Baizor-API 完全透明

唯一需要做的：1. common/model.go 新增 VideoGenerationModels 列表 2. 管理后台添加 huazhen-cluster Channel 3. 确保 Baizor-API 的视频 Task 适配器兼容 sulphur2 的响应格式

模块 6-8 ☑ 沿用

(同 plan20260708)

六、修订后的实施路线

Phase 0 已完成 ☑

Q1-Q5 已确认，无需额外验证周期。

Phase 1（本周起 ~ W8）：最小闭环

目标：用户通过 Web 工作台完成 剧本→分镜→图像/视频 全流程

周	任务	产出
W1	common/model.go 注册视频模型 + 管理后台配置 Channel	视频生成路由打通
W1-2	sulphur2 异步 API 封装（proxy 或 Baizor-API Task）	视频异步提交+轮询
W2-3	后端领域模型	StudioProject, StudioCharacter, StudioScene, StudioShot
W3-4	Studio API 端点	/api/studio/ CRUD + 快速生成 + Agent 创作
W3-5	快速生成 Service	studio_quick_gen.go（图像、视频、剧本拆解）
W4-6	Hermes Skill 编写	script-analyzer.md, shot-planner.md, batch-generator.md
W5-7	前端 Studio 页面	项目创建 + 角色管理 + 分镜列表（网格视图）
W6-7	快速生成 UI（复用 Image Playground 组件）	单镜头图像/视频生成面板
W7-8	Agent 创作 UI（复用 Hermes Playground WS）	Agent 进度和结果展示

Phase 1 验收标准： - ☑ 用户提交剧本，“快速拆解” → 3-10s 返回结构化镜头列表 - ☑ 用户选择”Agent 智能拆解” → 深度分析 → 详细镜头规划 - ☑ 每个镜头可一键生成图像（huayu-drama-4, ~30s） - ☑ 每个镜头可一键生成视频（huayu-drama-4, ~5-

30min，异步Task） - ☑ 所有生成走 BillingSession 正确计费 - ☑ 分镜板页面展示图像+视频

Phase 2（W9 ~ W14）：i2v + 资产库 + 角色一致性 + 分镜编辑器

周	任务	产出
W9-10	sulphur2 i2v 扩展	ComfyUI i2v 工作流 + API 封装
W9-11	资产库后端	StudioAsset + 对象存储
W11-13	角色一致性 Skill	character-designer.md + consistency-guard.md
W12-14	分镜编辑器 MVP	Konva.js Canvas

Phase 3（W15 ~ W20）：后期 + 审计 + 私有化

（同 plan20260708 Phase 3）

七、配置变更清单（可直接执行）

7.1 Baizor-API 代码变更

文件	变更	优先级
common/model.go	新增 VideoGenerationModels + IsVideoGenerationModel()	P0
common/model.go	确认 huayu-drama-4 同时在图像和视频列表中	P0

7.2 anne-1 服务器配置变更

文件	变更	优先级
docker-compose.yml	PROXY_MODELS 增加 huayu-drama-4	P0
proxy/proxy.py	模型名 huayu-drama-4 路由支持（所有路径）	P0
docker-compose.yml	SULPHUR2_NUM_FRAMES=193 (8秒视频)	P1
docker-compose.yml	SULPHUR2_NUM_INFERENCE_STEPS=75	P1
sulphur2/server.py	新增 /v1/videos/image-to-video 端点	P2
comfyui-workflows/	新增 ltx23_i2v_base_api.json	P2
proxy/proxy.py	新增异步任务模式（提交→轮询→取结果）	P1

7.3 管理后台配置

新增 Channel：
名称：huazhen-cluster
类型：OpenAI
Base URL：http://<anne-1-ip>:8080
Key：<LLAMA_API_KEY>
模型：huayu-drama-4
优先级：10
权重：100

八、风险更新

风险	等级	应对
API 规范不明确	☒ → ☑ 已确认	Q1-Q5 已全部确认
1920p + 193帧 OOM	☒	分步测试：先扩时长再扩分辨率
视频生成耗时 (15-40min)	☒	Task 异步 + 进度展示 + 允许用户取消
视频并发低 (当前 1)	☒	Phase 2 增加 ComfyUI 第二实例
i2v 质量不确定	☒	Phase 2 开始前先人工测试 ComfyUI i2v 效果
角色一致性效果	☒	多方案并行：LoRA + IP-Adapter + 后处理修复
分镜 Canvas 开发量	☒	Phase 1 用列表视图，Phase 2 Canvas

九、影视创作平台在「一人公司」下的组织方案

9.1 现状分析

一人公司的现有结构：

一人公司 (sidebar group: one-person-company)

- └ 市场部 → /one-person-company?dept=marketing → HermesAgentWorkspace (scoped chat)
- └ 运营部 → /one-person-company?dept=ops → HermesAgentWorkspace (scoped chat)
- └ 设计部 → /one-person-company?dept=design → HermesAgentWorkspace (scoped chat)
- └ 研发部 → /one-person-company?dept=rd → HermesAgentWorkspace (scoped chat)

本质：每个”部门”只是一个带不同 system prompt 的 Hermes 对话窗口，**无后端模型、无持久化项目、无状态流转**。会话存 localStorage。

影视创作平台需要什么： - 持久化的项目 → 角色 → 场景 → 镜头 树形结构 - 每个节点有状态（草稿 → 进行中 → 已生成 → 已审核） - 每个节点可触发 AI 生成（图像/视频） - 全局时间线视图 - 资产管理

差距：现有一人公司是”纯对话”，影视平台需要”项目管理 + AI 对话”。

9.2 三个方案对比

方案 A：「影视部」— 纯 Agent 对话（最轻量）

像现有部门一样，加一个 ?dept=film 的 Hermes 对话入口。

一人公司

- └ 市场部
- └ 运营部
- └ 设计部
- └ 研发部
- └ 影视部 ☒ → HermesAgentWorkspace + 影视 system prompt

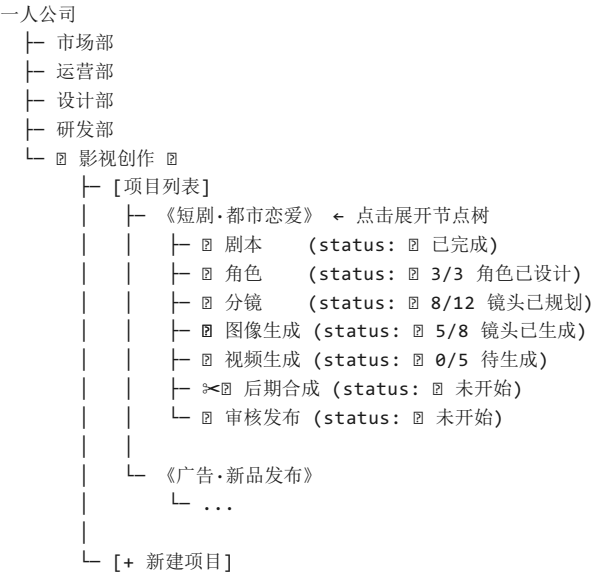
- Agent 在对话中管理项目（“帮我拆解这个剧本”、“生成第 3 个镜头”）
- 所有状态在对话上下文中，无结构化存储
- 靠 Agent 记忆维持项目进度

☒ 优点	☒ 缺点
零后端开发	无可视化分镜板
1 天上线	换个对话窗口项目就丢了
适合探索/原型	无法管理多个项目
	无法多人协作
	镜头状态全靠 Agent 记忆

适用场景：快速验证 Agent 编排能力，不适合作为产品。

方案 B：「项目节点树」— 创建项目后自动生成维度节点 ☒ 推荐

核心思路：在“一人公司”下新增“影视创作”入口，用户创建项目后，系统自动生成一棵生产流水线节点树，每个节点对应一个创作阶段，节点内可打开 Agent 对话或执行快速操作。



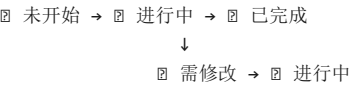
创建项目时自动生成的节点：

```
// 项目创建后自动生成 7 个阶段节点
var DefaultProjectStages = []StudioStage{
  {Key: "script",      Name: "剧本",      Order: 1, AutoSkill: "script-analyzer"},
  {Key: "characters",  Name: "角色",      Order: 2, AutoSkill: "character-designer"},
  {Key: "storyboard",  Name: "分镜",      Order: 3, AutoSkill: "shot-planner"},
  {Key: "image_gen",   Name: "图像生成", Order: 4, AutoSkill: "batch-generator"},
  {Key: "video_gen",   Name: "视频生成", Order: 5, AutoSkill: "batch-generator"},
  {Key: "post",        Name: "后期合成", Order: 6, AutoSkill: "post-production"},
  {Key: "review",      Name: "审核发布", Order: 7, AutoSkill: ""},
}
```

每个节点可以做什么：

节点	快速操作（Go 直调）	Agent 操作（Hermes）	节点产出
剧本	粘贴文本 → structured output 快速拆解	Agent 深度拆解 + 优化建议	场景列表 JSON
角色	输入描述 → 生成参考图	Agent 设计角色卡 + LoRA 参数	角色卡（图片+描述）
分镜	基于场景自动规划	Agent 优化景别/运镜/时长	镜头列表 + 参数
图像生成	每个镜头一键生成	Agent 批量生成 + 质量检查	关键帧图像
视频生成	每个镜头一键生成	Agent 批量 i2v + 重试	视频片段
后期合成	—	Agent 编排 FFmpeg 拼接	成片
审核发布	一键标记通过/驳回	—	状态变更

节点状态流转：



每个节点的状态由子项完成度自动计算：- 剧本节点：有输出 JSON → ☒ - 角色节点：所有角色有参考图 → ☒ - 图像生成节点：所有镜头有图像 → ☒，部分有 → ☒ (5/12)

后端模型：

```
// model/studio_stage.go
type StudioStage struct {
    Id          int    `json:"id" gorm:"primaryKey"`
    ProjectId   int    `json:"project_id" gorm:"index"`
    Key         string `json:"key" gorm:"type:varchar(32)"` // "script", "characters", ...
    Name        string `json:"name" gorm:"type:varchar(64)"`
    Order       int    `json:"order"`
    Status      int    `json:"status"` // 0=未开始 1=进行中 2=已完成 3=需修改
    AutoSkill   string `json:"auto_skill" gorm:"type:varchar(64)"` // 关联 Hermes Skill
    Progress    string `json:"progress" gorm:"type:text"` // JSON: {total:12, done:5}
    OutputData  string `json:"output_data" gorm:"type:text"` // JSON: 该阶段的结构化输出
    CreatedAt   int64  `json:"created_at" gorm:"autoCreateTime:milli"`
    UpdatedAt   int64  `json:"updated_at" gorm:"autoUpdateTime:milli"`
}
```

前端 UI 结构：

```
/one-person-company?dept=film           → 项目列表
/one-person-company?dept=film&project=123 → 项目看板（节点树 + 状态总览）
/one-person-company?dept=film&project=123&stage=storyboard → 分镜节点详情
```

或者独立路由：

```
/studio           → 项目列表
/studio/$projectId → 项目看板（节点总览）
/studio/$projectId/$stageKey → 节点详情（包含 Agent 对话面板）
```

☒ 优点	☒ 缺点
结构化项目管理，清晰的生产流水线	需要后端模型开发（StudioProject + StudioStage）
每个节点状态可追踪	前端需要新的项目看板 UI
快速操作 + Agent 混合	比方案 A 工作量大
自动生成节点，用户无需手动组织	
一人公司理念一致：一个人管全流程	
节点可自定义/扩展	

方案 C：「独立 Studio 模块」— 脱离一人公司

影视创作作为顶层 sidebar group，完全独立于一人公司。

```
Sidebar Product Rail:
├─ 概览 (overview)
├─ 工作台 (workbench)
├─ 一人公司 (one-person-company)
├─ ☐ 影视工作室 (studio) ☐ ← 独立入口
├─ 团队协作 (team-collaboration)
└─ ...
```

☒ 优点	☒ 缺点
独立空间，不受一人公司约束	产品定位分裂：一人公司 vs Studio
可以做更复杂的专业 UI	重复造轮子（用户管理、计费 etc）
适合未来独立产品化	侧边栏越来越拥挤

9.3 推荐方案：B — 项目节点树

理由：

- 1. 产品逻辑自治：一人公司 = 一个人做所有事。市场部管获客，运营部管交付，影视部管创作 — 自然延伸
- 2. 架构复用最大化：
 - 项目创建/管理 → 复用 Team 的 CRUD 模式
 - 节点内 Agent 对话 → 复用 HermesAgentWorkspace（传入 stage 对应的 Skill）
 - 图像/视频生成 → 复用 Image Playground 组件

- 异步任务 → 复用 `HermesExecutionTask`
3. **自动生成节点比手动管理好**：用户不需要了解”影视生产流水线有哪些阶段”，系统替他组织。专业用户可以跳过/自定义节点
4. **渐进式复杂度**：
- Phase 1：节点树 = 简单的阶段列表页，每个节点只有状态和一个”开始”按钮
 - Phase 2：节点内展开分镜网格、Canvas 编辑器
 - Phase 3：时间线、后期、审批

9.4 方案 B 详细设计

9.4.1 路由设计

```
# 方案 B1: 嵌入一人公司路由 (推荐)
/one-person-company?dept=film           → 影视创作首页 (项目列表)
/one-person-company?dept=film&project={id} → 项目看板
/one-person-company?dept=film&project={id}&stage={key} → 节点详情
```

```
# 方案 B2: 独立路由但从一人公司入口进入
/studio           → 项目列表
/studio/{projectId} → 项目看板
/studio/{projectId}/{stageKey} → 节点详情
```

推荐 B2，理由： - URL 更干净，便于分享和书签 - 路由参数用 TanStack Router 的 path params 更自然 - 侧边栏一人公司的”影视创作”菜单项链接到 `/studio` - 保持一人公司的归属关系 (sidebar 高亮一人公司)

9.4.2 Sidebar 集成

```
// use-sidebar-data.ts - 修改一人公司 group
{
  id: 'one-person-company',
  title: t('One-Person Company'),
  icon: Building2,
  iconColor: 'text-violet-500',
  url: '/one-person-company',
  position: 'top',
  items: [
    { title: t('Marketing department'), url: '/one-person-company?dept=marketing', icon: Megaphone },
    { title: t('Operations department'), url: '/one-person-company?dept=ops', icon: ListChecks },
    { title: t('Design department'), url: '/one-person-company?dept=design', icon: Palette },
    { title: t('R&D department'), url: '/one-person-company?dept=rd', icon: Code2 },
    // 🎬 影视创作
    { title: t('Film Studio'), url: '/studio', icon: Clapperboard },
  ],
}
```

`resolveActiveRootGroup` 新增映射： `/studio` → `one-person-company`

9.4.3 后端 API

```

# 项目 CRUD
GET    /api/studio/projects          - 项目列表
POST   /api/studio/projects          - 创建项目（自动生成 7 个阶段节点）
GET    /api/studio/projects/:id     - 项目详情（含所有阶段状态）
PUT    /api/studio/projects/:id     - 更新项目
DELETE /api/studio/projects/:id     - 删除项目

# 阶段节点
GET    /api/studio/projects/:id/stages - 获取所有阶段（含状态、进度）
PUT    /api/studio/projects/:id/stages/:key - 更新阶段（状态、输出数据）
POST   /api/studio/projects/:id/stages/:key/run - 触发阶段执行
body: { mode: "quick" | "agent" }
quick → 调 QuickGenService
agent → 创建 HermesExecutionTask

# 镜头（分镜阶段的子项）
GET    /api/studio/projects/:id/shots - 镜头列表
POST   /api/studio/projects/:id/shots - 创建/批量创建镜头
PUT    /api/studio/projects/:id/shots/:shotId - 更新镜头
POST   /api/studio/projects/:id/shots/:shotId/generate - 单镜头生成
body: { type: "image" | "video" | "i2v" }

# 角色
GET    /api/studio/projects/:id/characters
POST   /api/studio/projects/:id/characters
PUT    /api/studio/projects/:id/characters/:charId

```

9.4.4 后端模型

```

// model/studio_project.go
type StudioProject struct {
    Id          int    `json:"id" gorm:"primaryKey"`
    UserId      int    `json:"user_id" gorm:"index"`
    TeamId      int    `json:"team_id" gorm:"index" // 0=个人项目
    Name        string `json:"name" gorm:"type:varchar(128)"`
    Brief       string `json:"brief" gorm:"type:text" // 创意简介
    Genre       string `json:"genre" gorm:"type:varchar(32)" // short_drama, commercial, animation
    Status      int    `json:"status" gorm:"default:1" // 1=draft 2=in_progress 3=completed
    StyleDNA    string `json:"style_dna" gorm:"type:text" // JSON: 视觉风格参数
    CreatedAt   int64  `json:"created_at" gorm:"autoCreateTime:milli"`
    UpdatedAt   int64  `json:"updated_at" gorm:"autoUpdateTime:milli"`
}

// model/studio_stage.go - 创建项目时自动生成
type StudioStage struct {
    Id          int    `json:"id" gorm:"primaryKey"`
    ProjectId   int    `json:"project_id" gorm:"index;uniqueIndex:idx_project_key"`
    Key         string `json:"key" gorm:"type:varchar(32);uniqueIndex:idx_project_key"`
    Name        string `json:"name" gorm:"type:varchar(64)"`
    Order       int    `json:"order"`
    Status      int    `json:"status" gorm:"default:0" // 0=not_started 1=in_progress 2=completed 3=needs_revisi
    AutoSkill   string `json:"auto_skill" gorm:"type:varchar(64)"`
    TotalItems  int    `json:"total_items" gorm:"default:0"`
    DoneItems   int    `json:"done_items" gorm:"default:0"`
    OutputData  string `json:"output_data" gorm:"type:text" // JSON: 阶段结构化产出
    CreatedAt   int64  `json:"created_at" gorm:"autoCreateTime:milli"`
    UpdatedAt   int64  `json:"updated_at" gorm:"autoUpdateTime:milli"`
}

```

9.4.5 项目看板 UI（前端核心页面）

《短剧·都市恋爱》

[Agent 对话] [设置]

短剧 · 12 个镜头 · 创建于 2026-07-15

剧本

已完成

[查看]

角色

3/3

[查看]

分镜

8/12

[继续]

图像

5/8

[继续]

视频

未开始

[需先完成
图像生成]

后期

未开始

审核

最近动态

10:32 Agent 完成 3 个镜头的图像生成

10:15 手动修改了镜头 #5 的描述

09:48 Agent 完成剧本拆解, 生成 12 个镜头

节点间依赖关系（前端做，不需要后端 DAG）：

```
const STAGE_DEPENDENCIES: Record<string, string[]> = {
  script:    [],                // 无前置
  characters: ['script'],       // 需要先有剧本（才知道有哪些角色）
  storyboard: ['script'],       // 需要先有剧本
  image_gen: ['storyboard'],    // 需要先有分镜
  video_gen: ['image_gen'],     // 需要先有关键帧图像（i2v pipeline）
  post:      ['video_gen'],     // 需要先有视频片段
  review:    ['post'],         // 需要先有成片
}
```

未满足依赖的节点显示为灰色 + “需先完成 XX” 提示。

9.4.6 节点内详情页

以”分镜”节点为例：

← 返回看板

分镜规划

[快速生成]

[Agent 智能规划]

#	描述	景别	运镜	时长	状态
1-1	咖啡厅全景	全景	推	3s	
1-2	女主入场	中景	跟	2s	
1-3	对话特写	特写	固定	4s	
2-1	街道夜景	全景	移	3s	
...	...				

[全部生成图像]

[全部生成视频]

[导出镜头表]

Agent 对话

[展开/折叠]

> 请优化第 2 场的运镜方案

建议第 2-1 镜头改为“升”镜, 增强视觉冲击...

关键设计：节点详情页下方嵌入一个折叠式 Agent 对话面板，复用 HermesAgentWorkspace 组件，但作用域限定在该节点（system prompt 注入当前项目/阶段上下文）。

9.5 与纯聊天部门的关系

部门	交互模式	后端模型	数据持久化
市场部	纯对话	无	localStorage
运营部	纯对话	无	localStorage
设计部	纯对话	无	localStorage
研发部	纯对话	无	localStorage
影视创作 	项目看板 + 嵌入式对话	StudioProject + StudioStage + StudioShot	数据库

影视创作是一人公司的第一个”重”部门。未来如果其他部门也需要结构化管理（比如运营部管客户管线、设计部管设计项目），可以复用同样的”项目 + 阶段节点”模式。

9.6 实施路线（融入 Phase 1）

周	任务	产出
W1	后端：StudioProject + StudioStage 模型 + 自动生成逻辑	DB migration
W1-2	后端：/api/studio/ CRUD API	项目和阶段 CRUD
W2-3	前端：/studio 项目列表页	创建项目 → 自动生成节点
W3-4	前端：/studio/\$projectId 项目看板（节点卡片 + 状态 + 依赖）	流水线可视化
W4-5	前端：/studio/\$projectId/\$stageKey 节点详情 + 嵌入式 Agent 对话	节点内操作
W5-6	后端：分镜阶段的 Shot CRUD + 生成触发	镜头管理 + 生图/生视频
W6-7	前端：分镜节点的镜头列表 + 单镜头生成 UI	关键帧展示
W7-8	联调：节点状态自动计算 + 全流程跑通	端到端验证

十、下一步行动 (Action Items)

立即执行（本周）

- ☐ **P0:** common/model.go 新增 VideoGenerationModels，注册 huayu-drama-4
- ☐ **P0:** anne-1 docker-compose.yml 的 PROXY_MODELS 增加 huayu-drama-4
- ☐ **P0:** 两个 llama-server 都加 --alias huayu-drama-4
- ☐ **P0:** 管理后台添加 huazhen-cluster Channel
- ☐ **P0:** 验证 Baizor-API → proxy → sulphur2 视频生成端到端链路
- ☐ **P1:** anne-1 试调 SULPHUR2_NUM_FRAMES=193，测试 8s 视频是否 OOM
- ☐ **P1:** 起草 StudioProject + StudioStage Go 模型文件

下周

- ☐ sulphur2 异步 API 封装（选定方案 A 或 B 后实施）
- ☐ Studio API 端点开发
- ☐ 前端 /studio 路由 + 项目列表页
- ☐ 起草 script-analyzer.md Skill 初稿
- ☐ sidebar 一人公司 group 加”影视创作”入口